

Bayesian Optimization Algorithm

Lucas Cavadini, Martin Calsina y Sebastian Souto

2026-05-14

Imaginemos este problema

Una empresa petrolera está buscando la mejor zona para perforar.
Pero surgen algunos inconvenientes:

- ▶ cada perforación cuesta mucho dinero
- ▶ lleva mucho tiempo
- ▶ solo vamos a conocer información sobre ese punto específico donde fue la perforación

El problema que estamos queriendo resolver es:

$$\max_{x \in A} f(x)$$

Funciones de caja negra

La función real $f(x)$ (en el ejemplo, cantidad de petróleo en el terreno en una ubicación x) es desconocida. Solo podemos probar un valor de x y observar el resultado.

Esto se conoce como función de **caja negra**:

- ▶ no conocemos su fórmula explícita
- ▶ no conocemos nada acerca de su forma (concavidad, linealidad)
- ▶ no tenemos las derivadas

Si quisiéramos encontrar un óptimo de la función objetivo de manera eficiente no podríamos usar métodos conocidos como, por ejemplo, descenso por el gradiente.

¿Cómo podemos encontrar las mejores zonas para perforar?

Una estrategia podría ser, por ejemplo, probar ubicaciones al azar. Pero esto puede ser muy ineficiente (cada perforación es cara, el presupuesto es limitado y encima podríamos quedarnos buscando en zonas malas).

Para abordar estos casos es que surgen métodos como la

Optimización Bayesiana:

- Propone una estrategia iterativa para hallar óptimos mediante el cual, a partir de observaciones iniciales, un modelo va aprendiendo sobre la distribución del conjunto de posibles soluciones. Se enfoca en hallar máximos **globales**.

Bayesian Optimization (BO)

- ▶ Construye un modelo probabilístico para estimar la distribución del espacio de soluciones factibles.
- ▶ Decide en qué puntos es mejor volver a evaluar a nuestra función objetivo.
- ▶ Con las nuevas observaciones se actualiza el modelo probabilístico y se repite el proceso.

Componentes principales

La optimización bayesiana tiene dos componentes fundamentales:

1. Surrogate model

Modelo probabilístico de la función objetivo.

En nuestro caso:

- ▶ Procesos Gaussianos.

2. Función de adquisición

Usa el surrogate model para decidir:

“¿Dónde conviene evaluar la función ahora?”

Ejemplos:

- ▶ Expected Improvement (EI)
- ▶ Upper Confidence Bound (UCB)

1. Procesos Gaussianos

Motivación

No conocemos la función verdadera:

$$f(x)$$

Entonces modelamos incertidumbre sobre ella.

Idea:

En vez de asumir una única función, consideramos una distribución de probabilidad sobre funciones posibles.

Definición formal

Un Proceso Gaussiano es una colección de variables aleatorias tal que cualquier subconjunto finito de ellas tiene distribución normal multivariada.

Notación:

$$f(x) \sim GP(\mu(x), k(x, x'))$$

Componentes

Mean function

$$\mu(x) = \mathbb{E}[f(x)]$$

Representa el valor esperado de la función.

Kernel (covariance function)

$$k(x, x') = \text{Cov}(f(x), f(x'))$$

Determina:

- ▶ correlación entre puntos,
- ▶ suavidad,
- ▶ periodicidad,
- ▶ comportamiento de las funciones.

Intuición:

Si $k(x, x')$ es grande cuando $x \approx x'$ entonces el GP “cree” que $f(x) \approx f(x')$

- ▶ Ejemplo: Kernel RBF

$$k(x, x') = \sigma_f^2 \exp\left(-\frac{\|x - x'\|^2}{2l^2}\right)$$

Hiperparámetros:

- ▶ l : length-scale
- ▶ σ_f^2 : varianza

Actualización Bayesiana del GP

Prior

Antes de observar datos:

$$f \sim GP(\mu, k)$$

Datos observados

Supongamos que observamos $D = \{(x_i, y_i)\}_{i=1}^n$ con $y_i = f(x_i)$ (o con ruido).

Posterior

Luego de observar datos:

$$f|D \sim GP(\mu_n, k_n)$$

El posterior sigue siendo un GP.

Distribución conjunta

Para puntos $x_1, \dots, x_n$, el GP induce:

$$(f(x_1), \dots, f(x_n), f(x_*)) \sim \mathcal{N}(\mu, K)$$

con:

$$K_{ij} = k(x_i, x_j)$$

Consideramos:

$$\begin{pmatrix} y \\ f_* \end{pmatrix} \sim \mathcal{N} \left(\begin{pmatrix} \mu(X) \\ \mu(x_*) \end{pmatrix}, \begin{pmatrix} K(X, X) & K(X, x_*) \\ K(x_*, X) & k(x_*, x_*) \end{pmatrix} \right)$$

Entonces:

$$f(x_*)|D \sim \mathcal{N}(\mu_n(x_*), \sigma_n^2(x_*))$$

► Posterior mean

$$\mu_n(x_*) = \mu(x_*) + K(x_*, X)K(X, X)^{-1}(y - \mu(X))$$

► Posterior variance

$$\sigma_n^2(x_*) = k(x_*, x_*) - K(x_*, X)K(X, X)^{-1}K(X, x_*)$$

2. Función de adquisición

Función que utiliza el modelo probabilístico actual para decidir cuál es el próximo punto más conveniente para evaluar de la función objetivo.

Trade-off exploración vs explotación

- ▶ Explotación

Evaluar donde la media predicha es alta.

- ▶ Exploración

Evaluar donde la incertidumbre es grande.

Upper Confidence Bound (UCB)

Definición

$$UCB(x) = \mu_n(x) + \kappa\sigma_n(x)$$

Interpretación

- ▶ Primer término $\mu_n(x) \rightarrow$ explotación.
- ▶ Segundo término $\kappa\sigma_n(x) \rightarrow$ exploración.

Expected Improvement (EI)

Definición

Sea

$$f^+ = \max_i y_i$$

el mejor valor observado. Definimos:

$$I(x) = \max(f(x) - f^+, 0)$$

Entonces:

$$EI(x) = \mathbb{E}[I(x)]$$

- Intuición: EI representa la mejora esperada respecto al mejor valor observado hasta el momento. El próximo punto a evaluar será aquel que maximice $EI(x)$.

Pseudocódigo del algoritmo

Definir prior GP sobre f

Observar f en n_0 puntos iniciales

$n \leftarrow n_0$

Mientras $n \leq N$:

 Actualizar posterior GP

$x_n \leftarrow \operatorname{argmax} a(x)$

$y_n \leftarrow f(x_n)$

 Agregar (x_n, y_n)

$n \leftarrow n + 1$

Retornar:

- mejor punto observado, o
- punto con mayor media posterior

Simulación

Benchmarks

Podemos comparar el rendimiento del algoritmo con otros dos más simples sobre un intervalo $[a, b]$ fijo y muchas funciones definidas sobre el mismo:

- ▶ Grid Search

Agarra N puntos equiespaciados en el intervalo, evalúa y toma el máximo.

- ▶ Random Search

Genera N puntos en el intervalo, evalúa y toma el máximo.

Regret

A fines de tener una métrica cuantitativa para la comparación, dada una función $f(x)$ con máximo en x_0 y x^* el óptimo encontrado por un algoritmo, definimos el *Regret* del mismo:

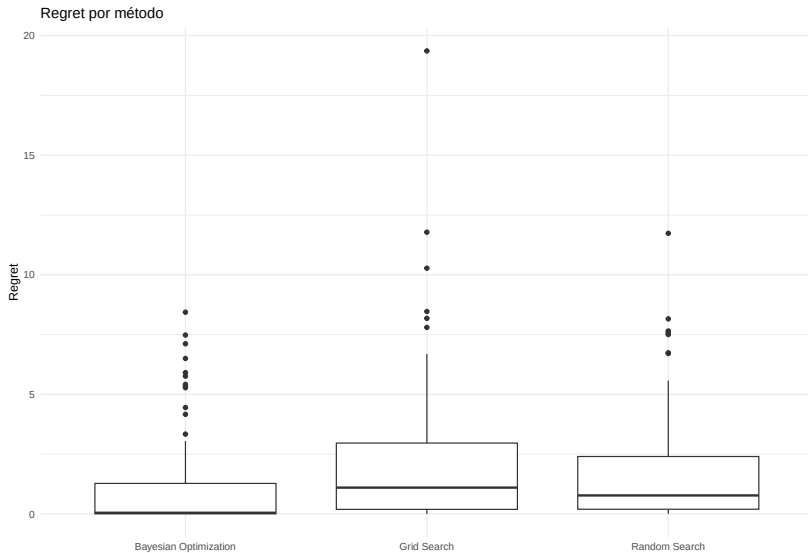
$$f(x_0) - f(x^*)$$

En lo siguiente, elegimos utilizar $[a, b] = [0, 100]$, $k = 100$ funciones y $N = 8$. En el caso de BO, N va a ser la cantidad total de sampleos.

Resultados

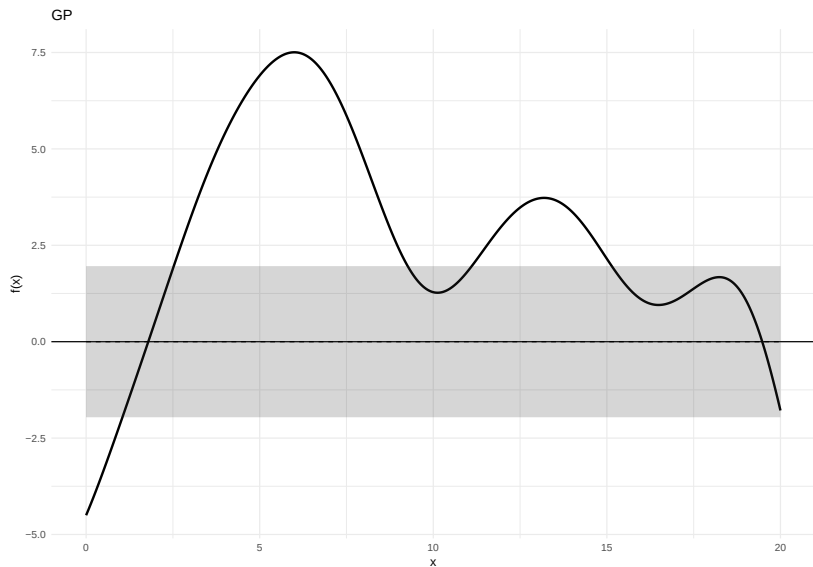
method	Regret promedio	Desvío regret	Mejor valor promedio
Bayesian Optimization	1.05	1.95	7.63
Random Search	1.69	2.23	6.99
Grid Search	2.16	3.03	6.53

Resultados

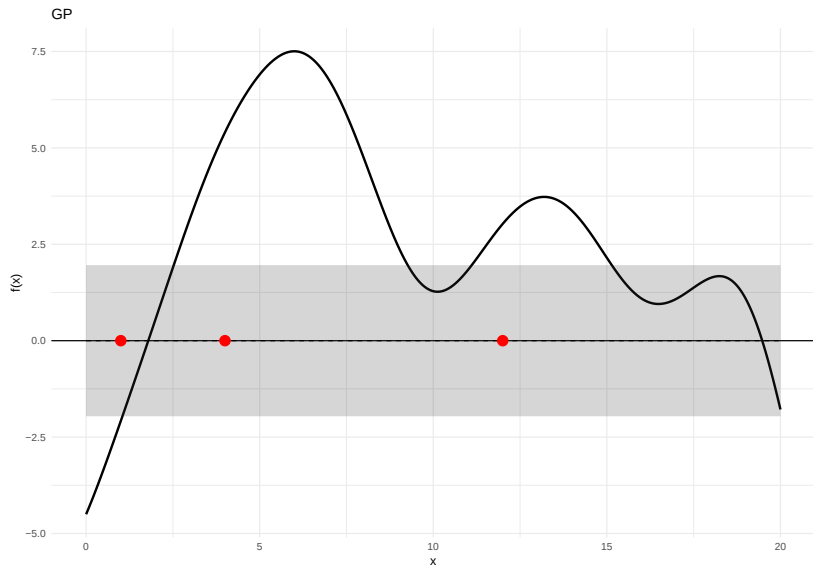


Ejecución paso a paso

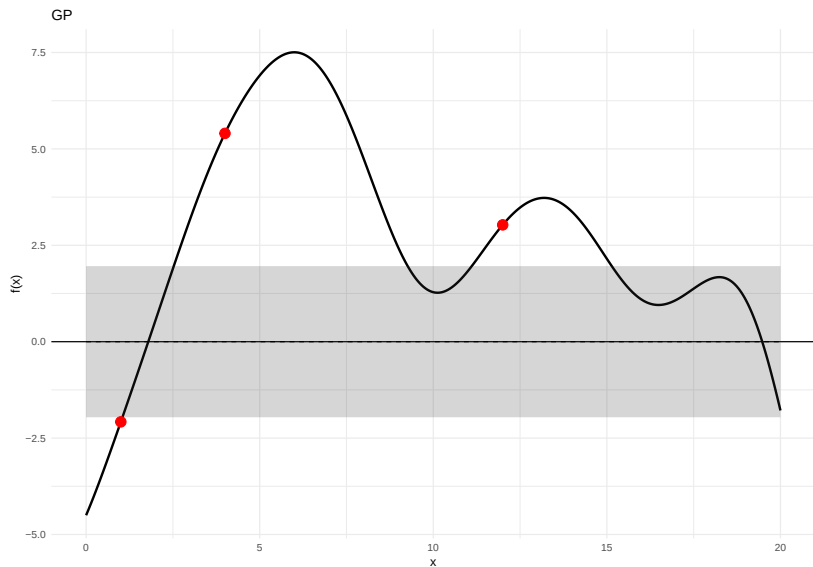
Función subyacente y GP a priori



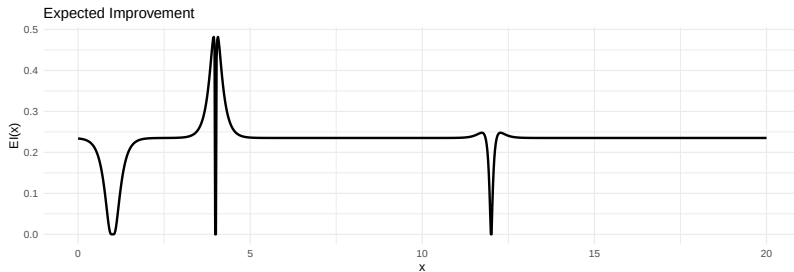
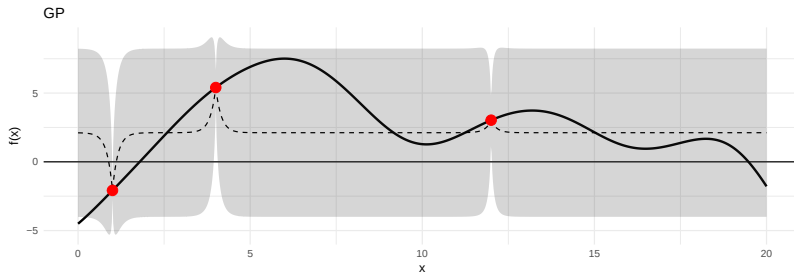
Sampleo inicial



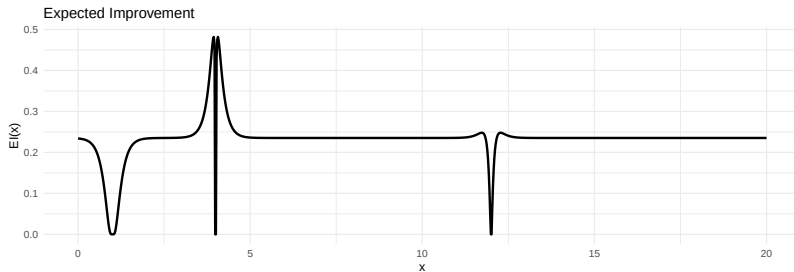
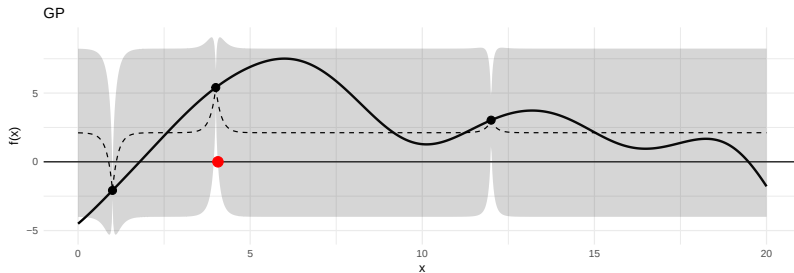
Evaluamos $f(x)$



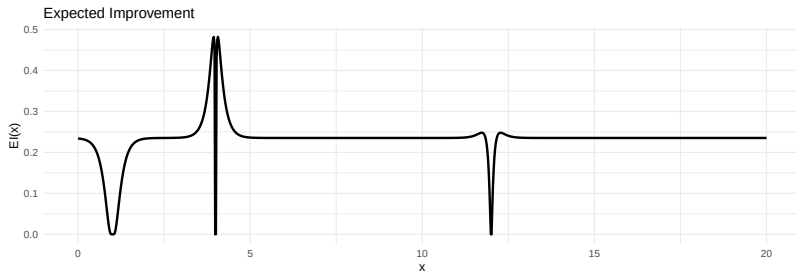
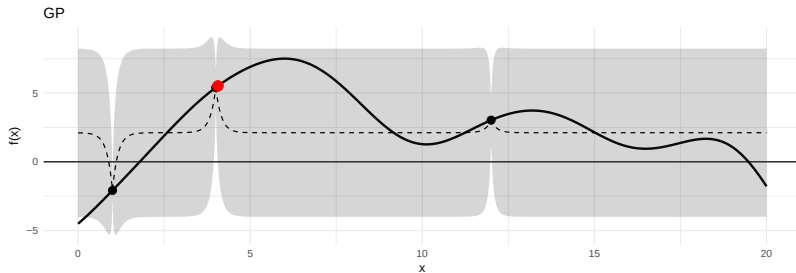
Actualizamos nuestro conocimiento



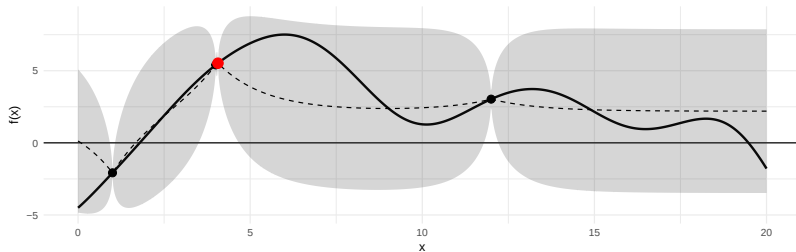
Elegimos un x^* que maximice $EI(x^*)$



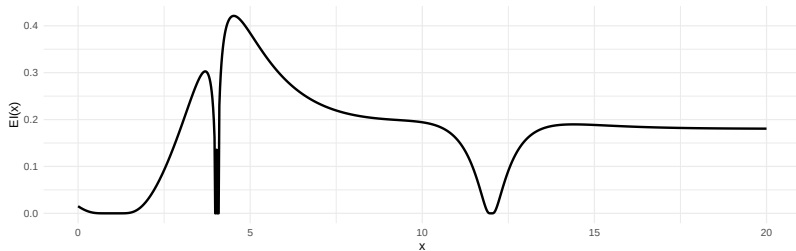
Evaluamos $f(x^*)$



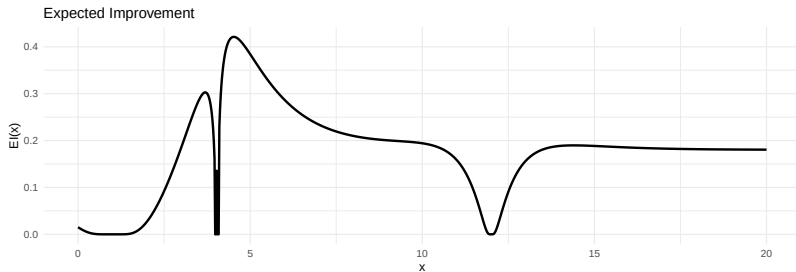
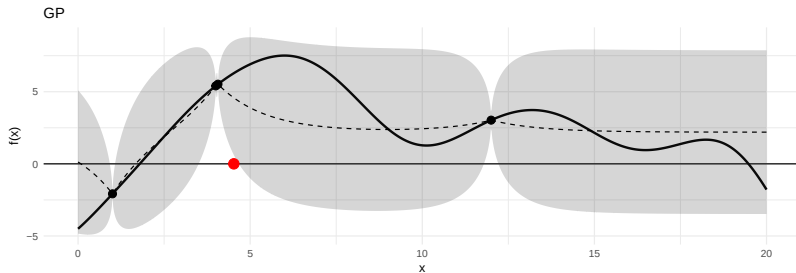
Actualizamos nuestro conocimiento



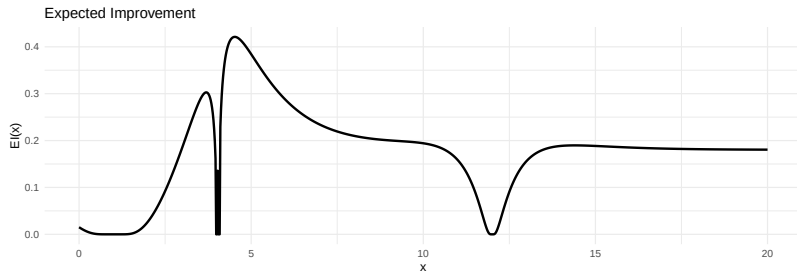
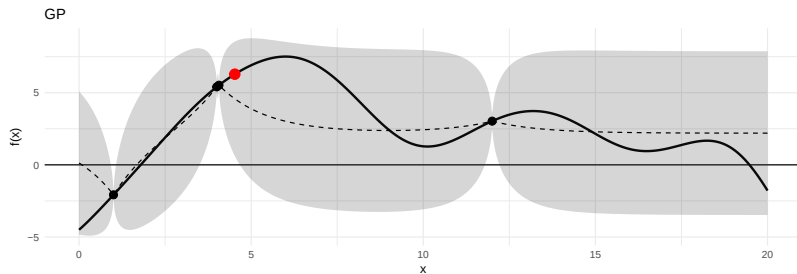
Expected Improvement



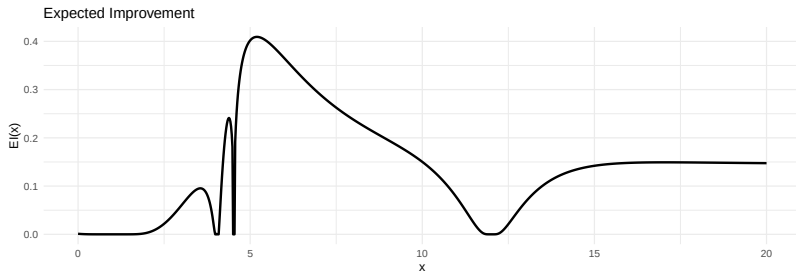
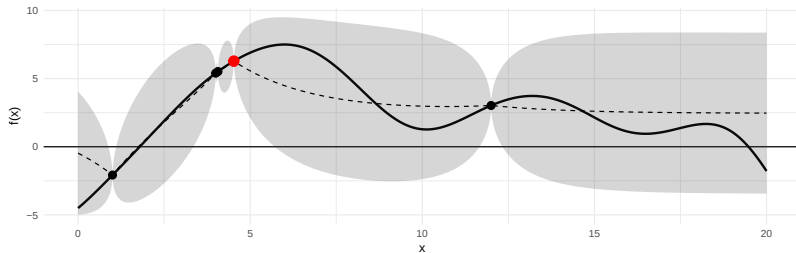
Elegimos un x^* que maximice $EI(x^*)$



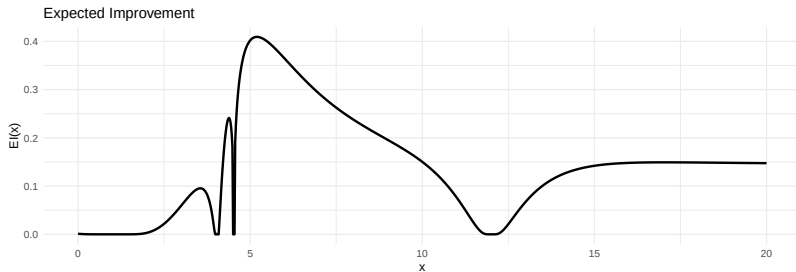
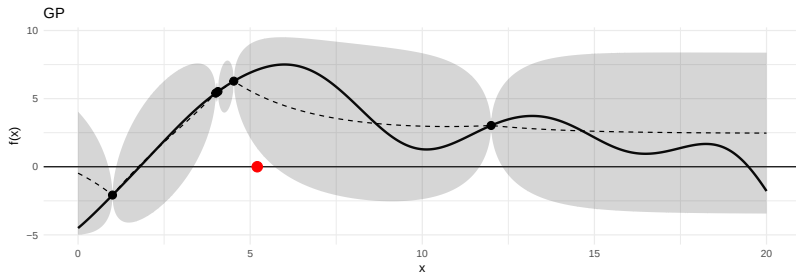
Evaluamos $f(x^*)$



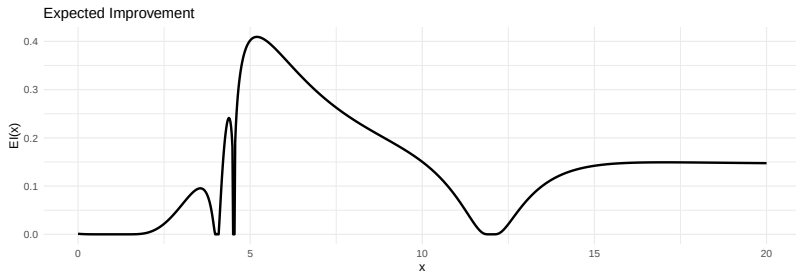
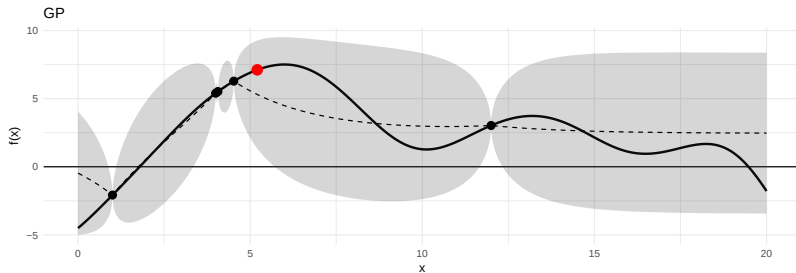
Actualizamos nuestro conocimiento



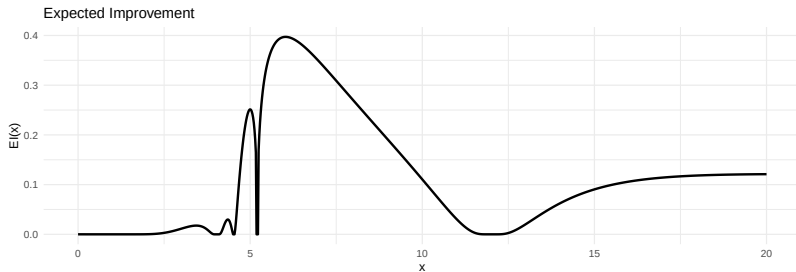
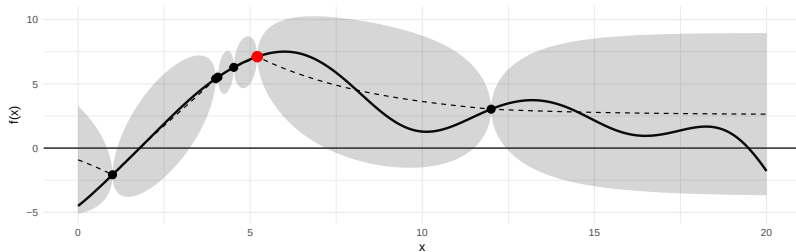
Elegimos un x^* que maximice $EI(x^*)$



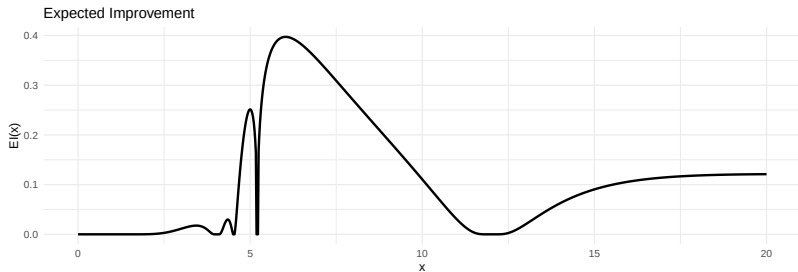
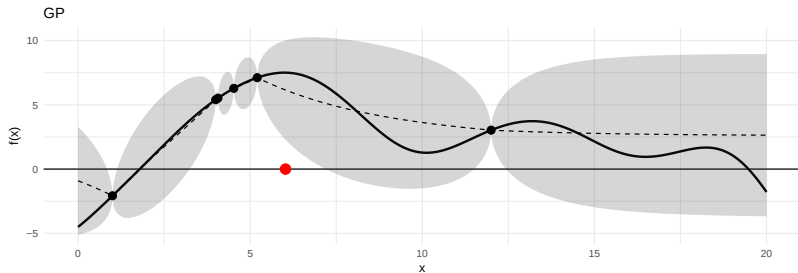
Evaluamos $f(x^*)$



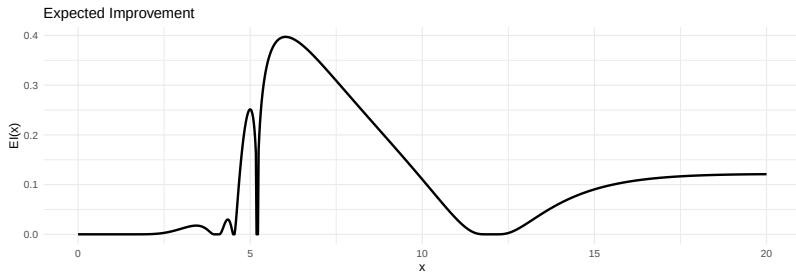
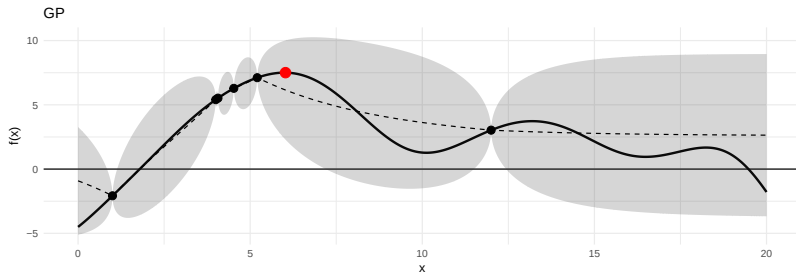
Actualizamos nuestro conocimiento



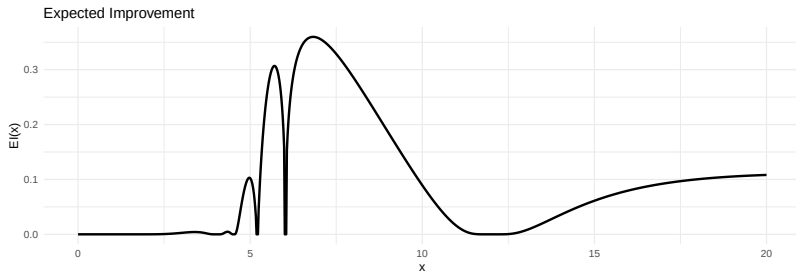
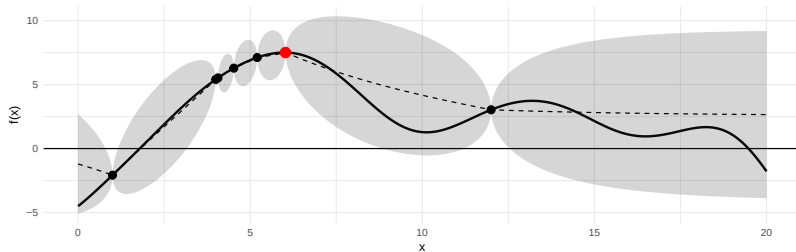
Elegimos un x^* que maximice $EI(x^*)$



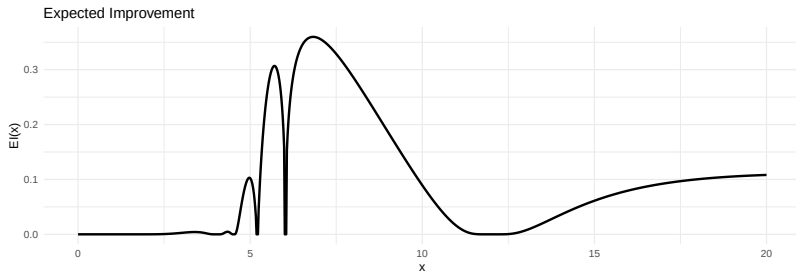
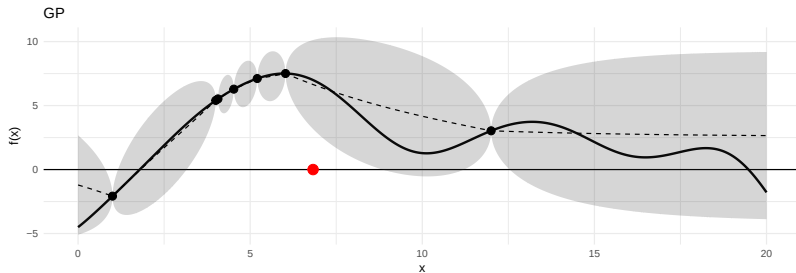
Evaluamos $f(x^*)$



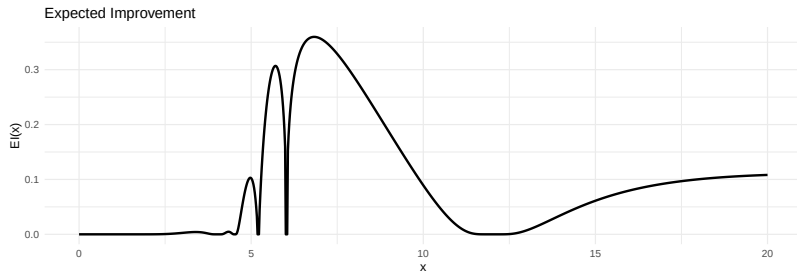
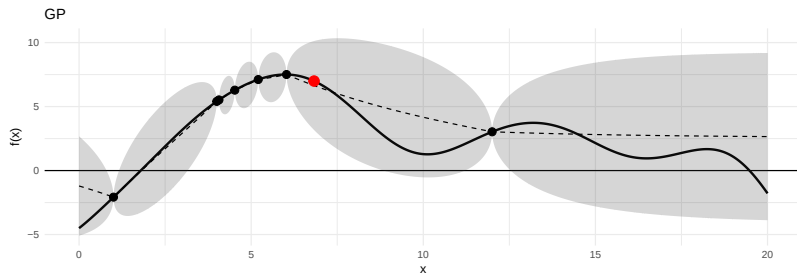
Actualizamos nuestro conocimiento



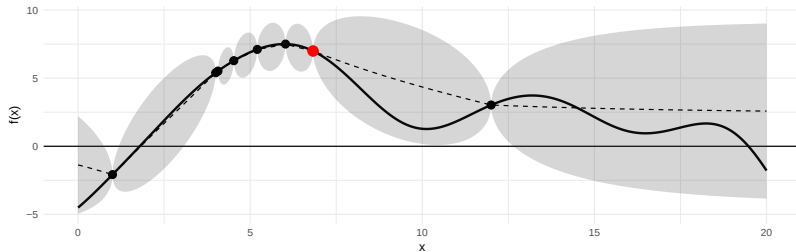
Elegimos un x^* que maximice $El(x^*)$



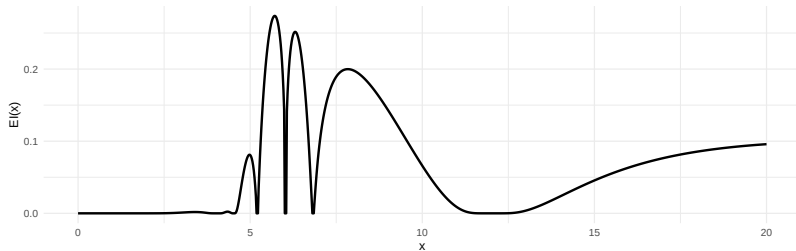
Evaluamos $f(x^*)$



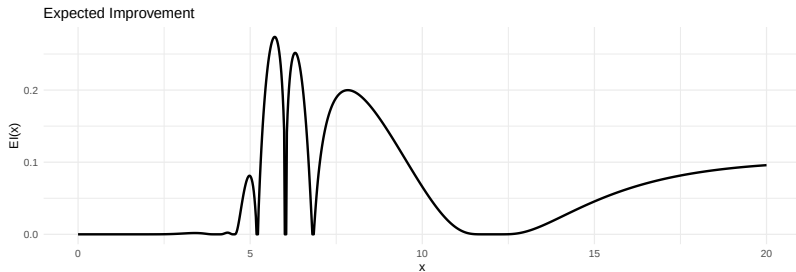
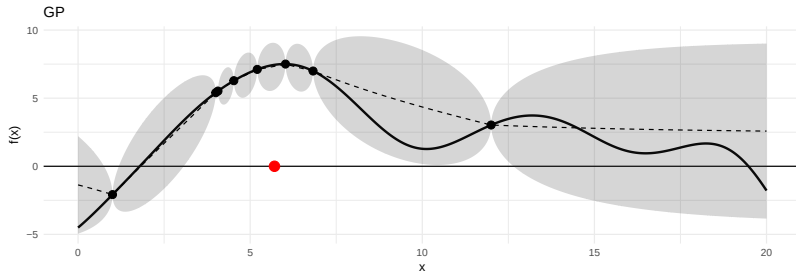
Actualizamos nuestro conocimiento



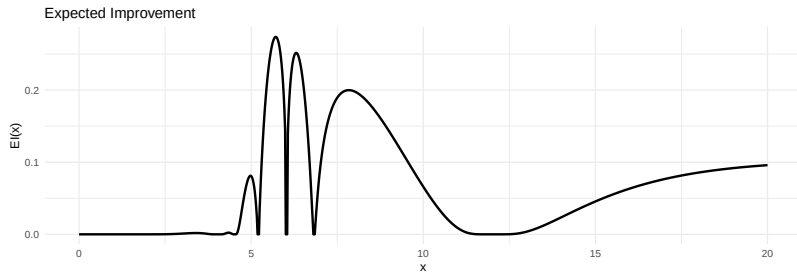
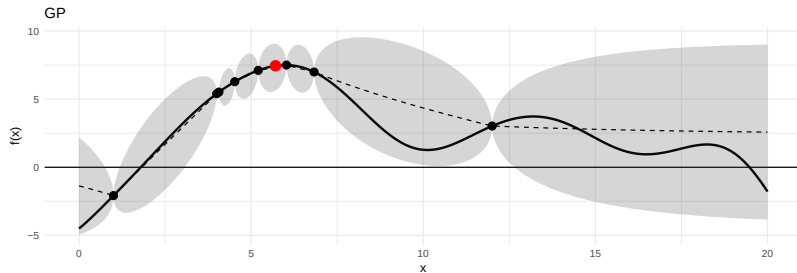
Expected Improvement



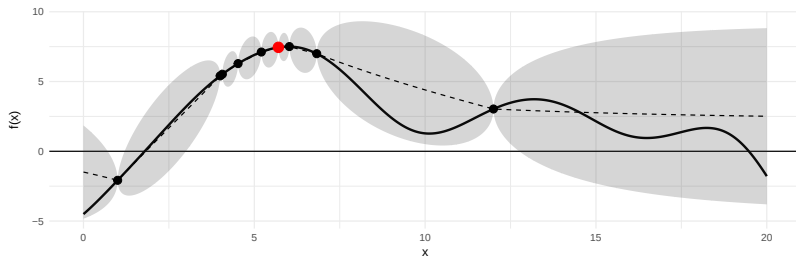
Elegimos un x^* que maximice $EI(x^*)$



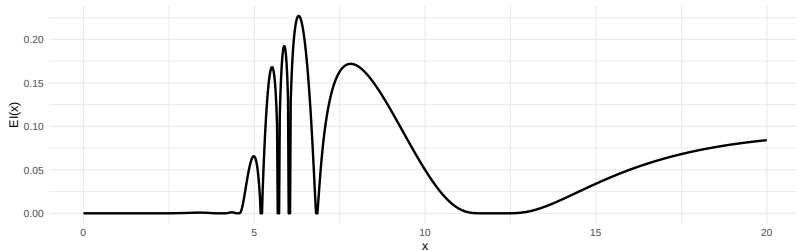
Evaluamos $f(x^*)$



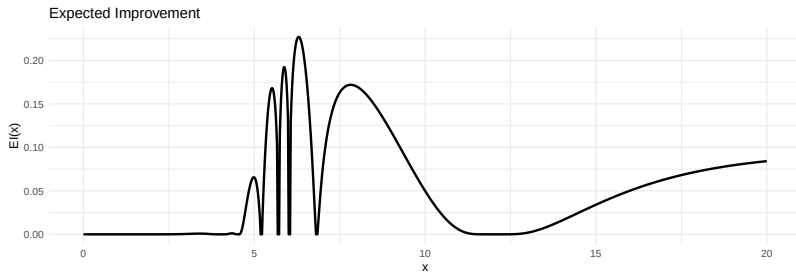
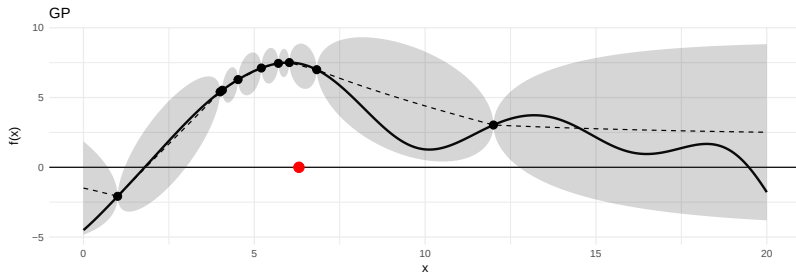
Actualizamos nuestro conocimiento



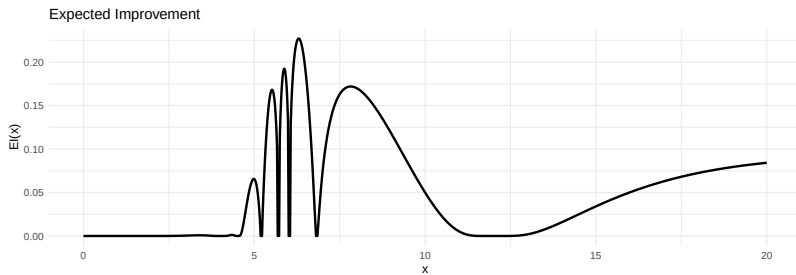
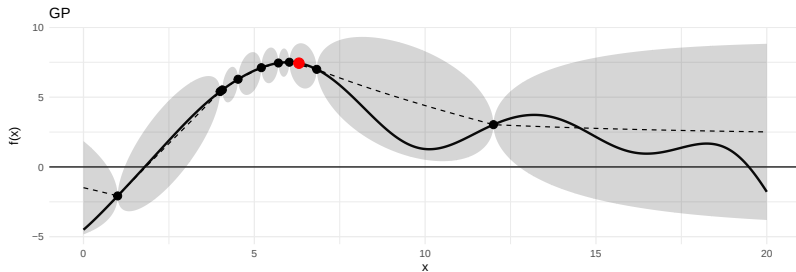
Expected Improvement



Elegimos un x^* que maximice $EI(x^*)$



Evaluamos $f(x^*)$



Cerrando

El algoritmo de Optimización Bayesiana puede resultar útil cuando:

- ▶ $f(x)$ es muy cara de evaluar.
- ▶ $f(x)$ es una caja negra.
- ▶ Tenemos cierto conocimiento o intuición sobre *cómo* es la función y queremos incorporarlo a nuestro algoritmo (en forma de *prior*).

Pero puede sufrir cuando:

- ▶ d la dimensión de x es muy grande (en general, $d > 20$).
- ▶ la función no es suave o tiene mucho ruido.